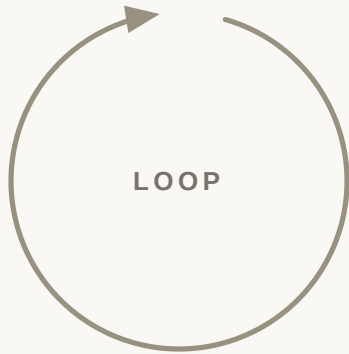


GFMI · MODEL RISK AND AI GOVERNANCE

Human in the Loop Is Not a New Pattern

Nicholas J. Goble, Ph.D. · Domino Data Lab

What do we actually mean by human in the loop?



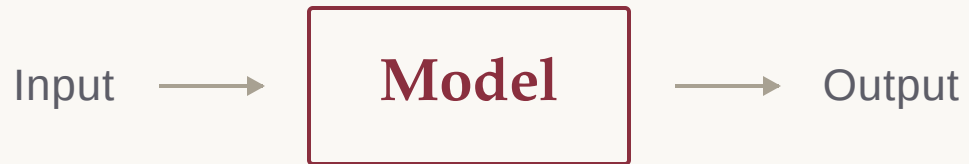
When do we add a human?

Where do we add a human?

And the question underneath both of them: is any of this new, or does it only feel new because we started calling it AI?

What is a model?

A model is an operator. Something goes in, something comes out.



And the reason we put a human in the loop at all is to manage the risk that the thing in the middle is wrong.

What is a human?

An entity that can be held accountable.

So the accurate name is

~~accountable entity in the loop.~~

Human in the loop sounds better, so that's the one that stuck. Keep the real definition though, because it's the accountability doing the work here, not the species.

Building and using have merged.

**We use models to
build models.**

**We use agents to
build agents.**

Which used to be two phases with a gate in between. Your controls still assume it is. With agents, the development loop is the usage loop, so there is no between for a gate to sit in.

THE EXAMPLE THAT IS ALREADY INSIDE YOUR BANK

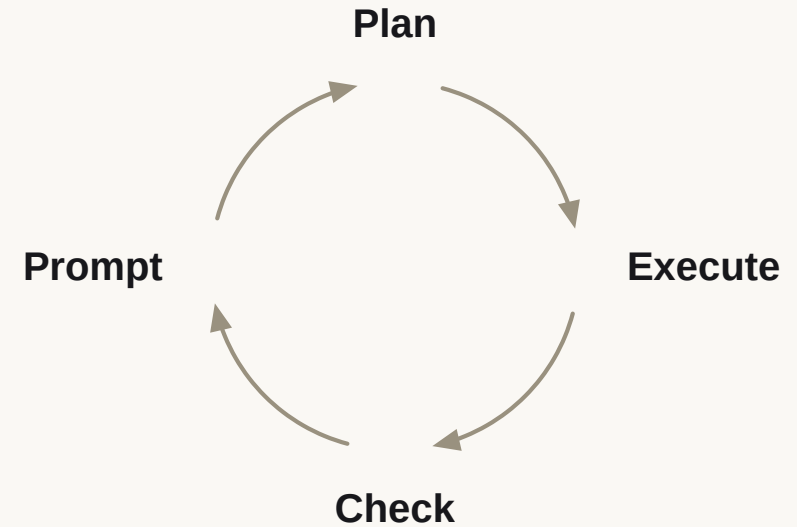
Vibe coding.

Show of hands. Who here has vibe coded?

From the outside, it looks like this.

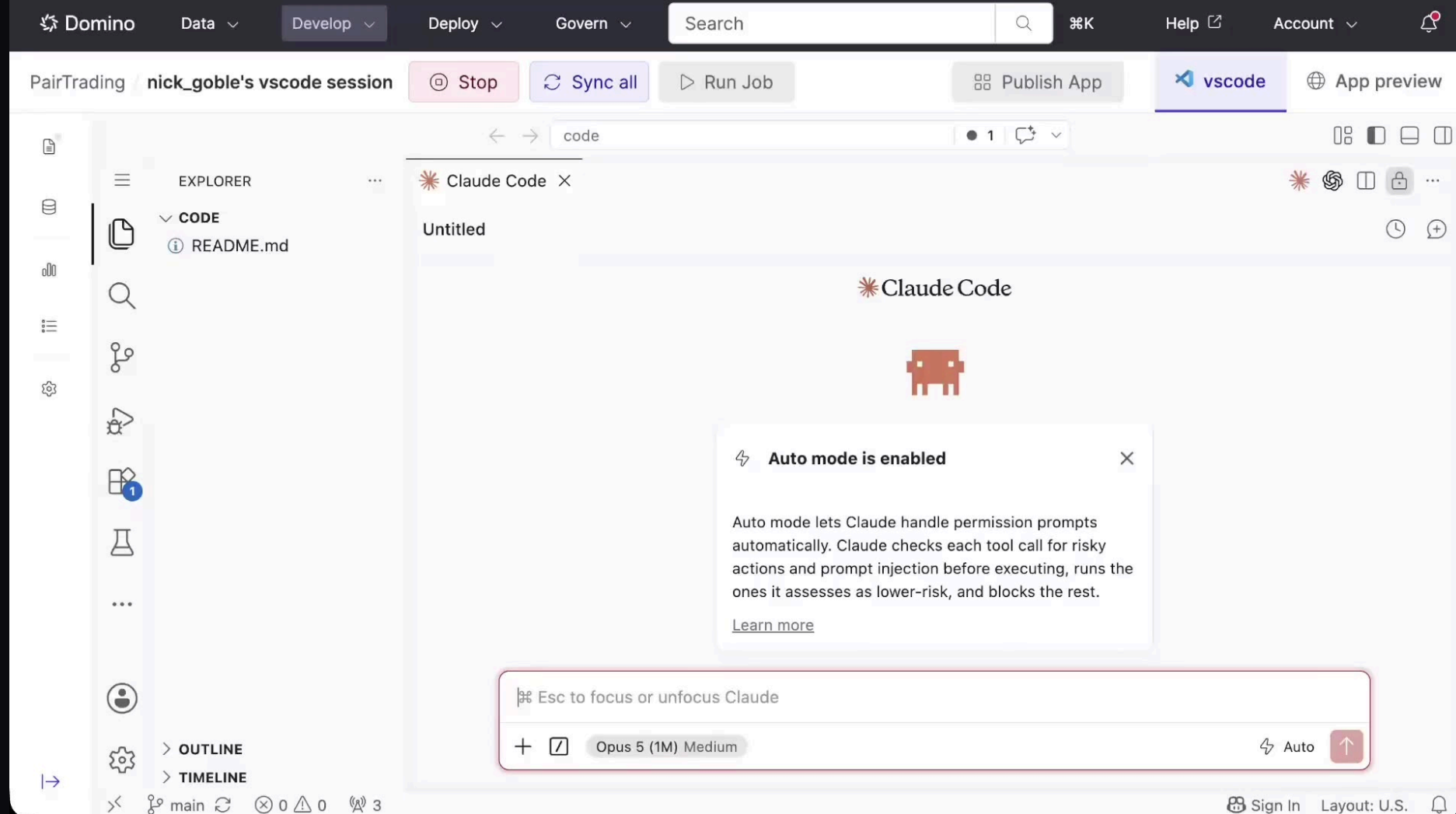


NO PROCESS ANYONE CAN POINT AT



WHAT IS ACTUALLY GOING ON

It isn't chaos. It's a loop, running fast enough that you can't see the individual turns.

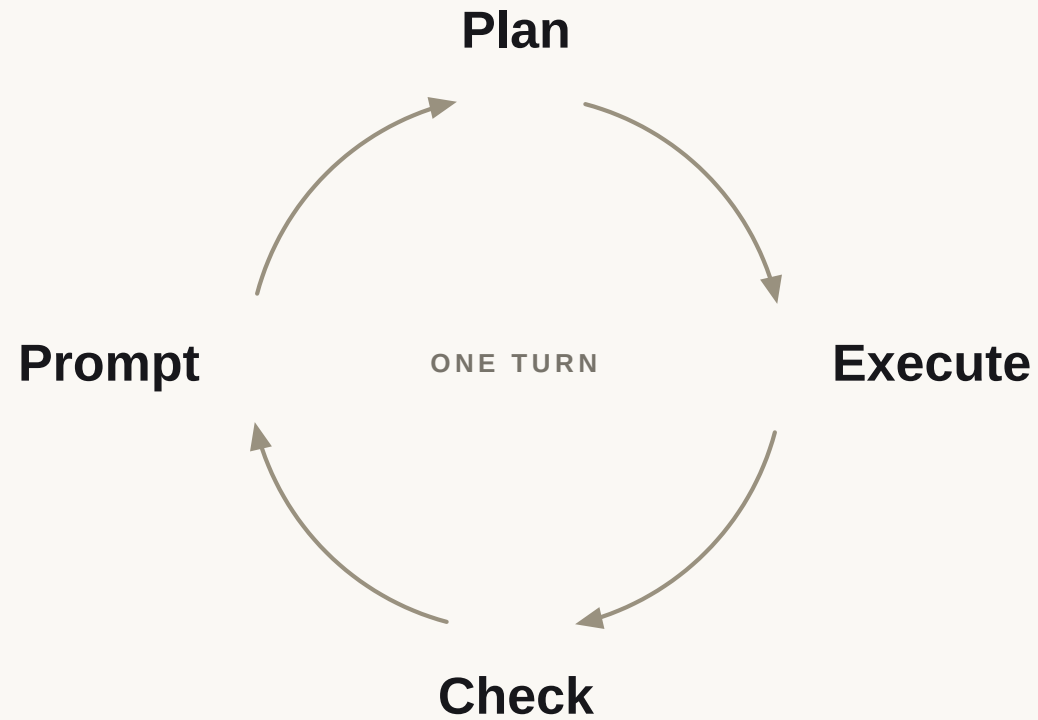


DEMO

Vibe coding, live

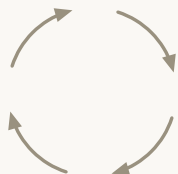
Watch for the turn: prompt, plan, execute, check. Every one ends with somebody deciding whether to accept what came back, and nobody writes that down.

This is all vibe coding is.



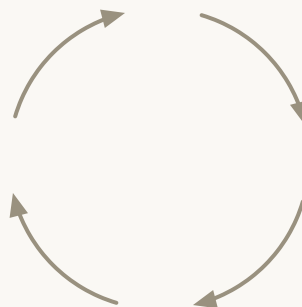
And each turn keeps getting bigger.

TWO YEARS AGO



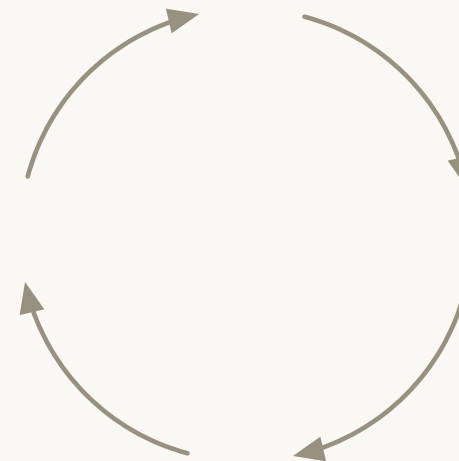
A function.
One file.

TODAY



A feature. A few files
and a test.

NEXT YEAR



A service.
Spec through to deploy.

Each turn covers more ground than it used to, so an unreviewed turn carries more than it used to. A control that reviews a fixed share of turns gets weaker every quarter without anybody touching it.

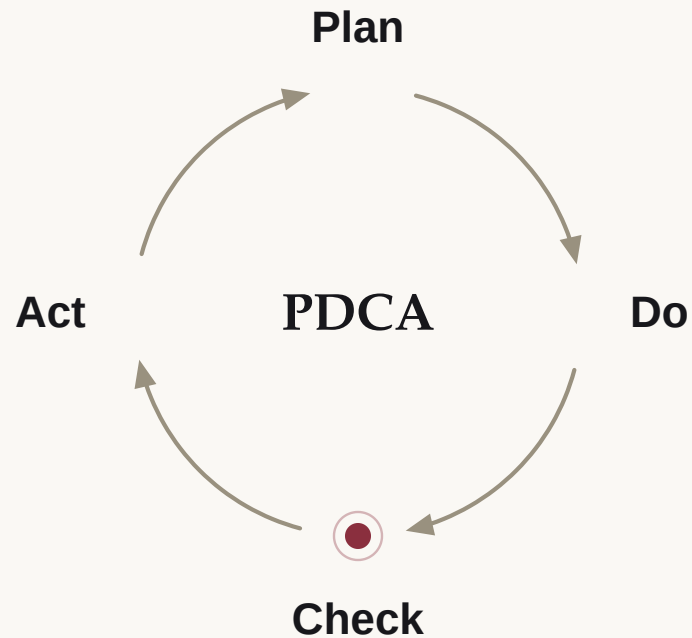
The loop isn't new.

This is iterative design.

This is a continual
improvement process.

Both have a control literature that predates
every model you own.

IT ALREADY HAS A NAME



PLAN

Decide what you're going to try, and what you expect to happen.

DO

Run it. Small, and on purpose.

CHECK

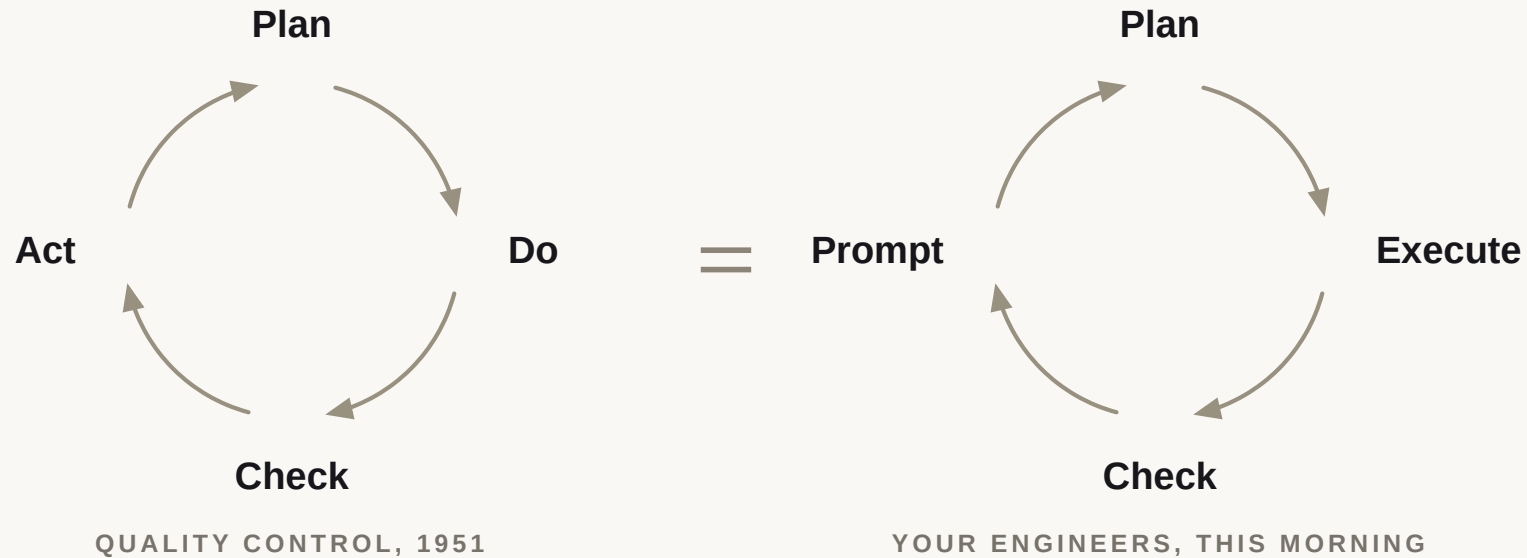
Compare what happened to what you said would happen.

ACT

Keep it, kill it, or change it. Then go around again.

Traces to Walter A. Shewhart, *Statistical Method from the Viewpoint of Quality Control* (1939), p. 45, a book Deming edited, where the cycle has three steps: specification, production, inspection. Deming took a modified version to a JUSE seminar in Tokyo in 1950 and Japanese practitioners recast it as the four-step Plan–Do–Check–Act in 1951. **Deming never accepted that name**, objecting that the English “check” means to hold back, and published his own Plan–Do–Study–Act cycle in *The New Economics* (1993).

It's the same loop.



Plan → Plan

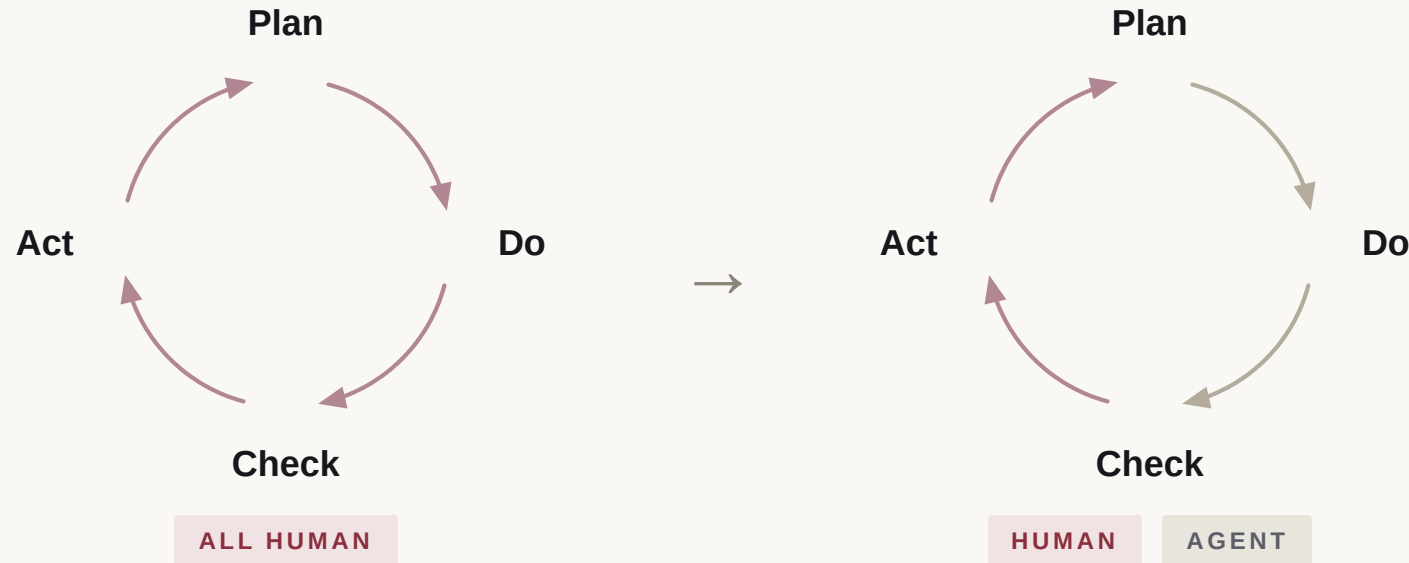
Do → Execute

Check → Check

Act → Prompt again

So this isn't a new control problem. It's a control problem with nearly ninety years of practice behind it, and you already own the framework.

What changed is who holds which step.



The agent took Plan and Do because that's the half that got cheap. Nobody has made Check and Act cheap, because those are the steps somebody has to answer for.

Accountability has to stay in the loop. The question is where.

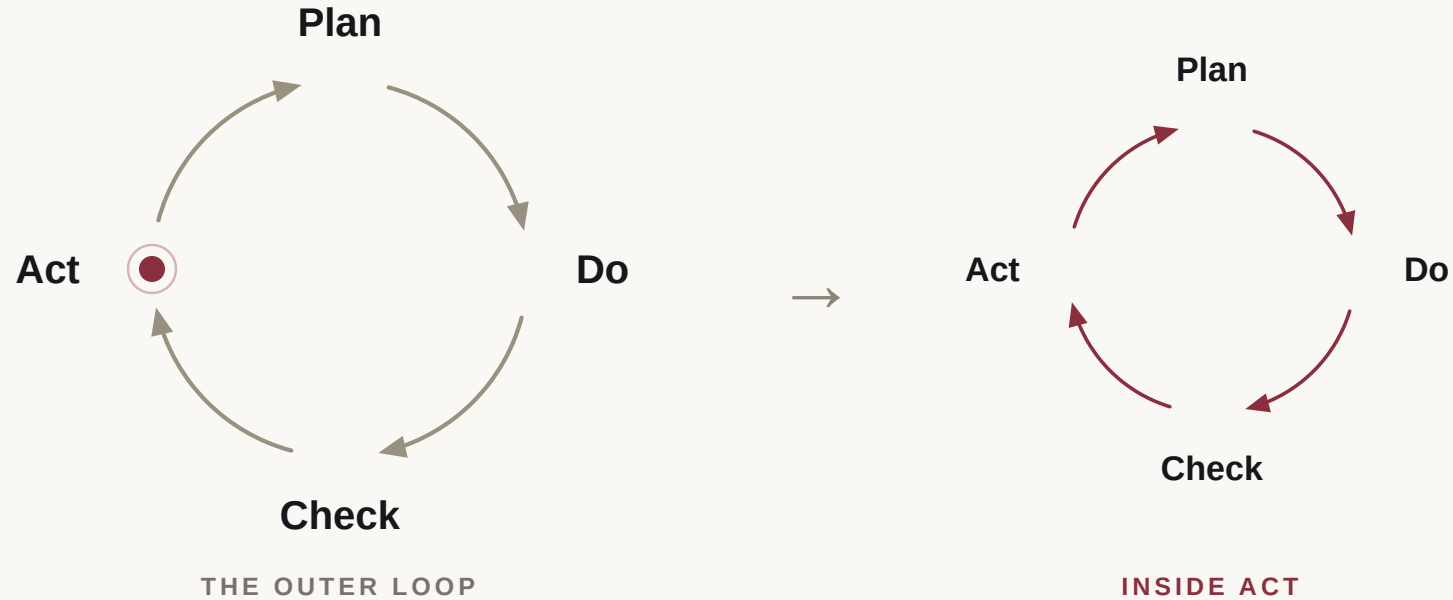


Check.

It's the only step where somebody has to accept the result. The rest of the loop is doing work. Check is where the work gets accepted.

An agent can do the work of a check. It can't be the thing that answers for the acceptance. So if the agent accepts as well, there's nobody left in the loop to hold accountable.

And Act is a loop of its own.



When a check fails, what happens next is itself a plan, a do, a check and an act.
That's your remediation process, and it needs the same discipline as the loop
it's fixing.

So what are the required checks?

FIRST LINE

CONTROL

Guardrails. The check that runs whether or not anybody is watching.

Owned by the people who build it and run it.

SECOND LINE

VALIDATE

Independent testing. Somebody who didn't build it forms their own expectation and tests against it.

Owned by a function with its own reporting line.

THIRD LINE

AUDIT

Documentation and audit readiness. Evidence the first two happened, and that the expectation was recorded before the output existed.

Owned by internal audit, reporting to the board.

Current form: IIA Statement of Position, *Three Lines Model: Assurance and Advice in Support of Effective Governance* (July 2026), which supersedes *The IIA's Three Lines Model* (July 2020) and the original position paper *The Three Lines of Defense in Effective Risk Management and Control* (January 2013). For US banks the doctrine also runs through the OCC Heightened Standards, 12 CFR 30 Appendix D. **Control, validate, audit are my labels, not the IIA's.**

So why does all of this feel new?

ONE

Your inventory now holds **indeterministic** models.
Same input, same version, different output, same
afternoon.

TWO

And the **use case isn't decided up front** any more.
You used to approve a model for a purpose. Now you
approve a tool, and the purpose gets picked at runtime
by whoever is holding it.

THE GUIDANCE YOU WOULD REACH FOR

Generative AI and agentic AI models are novel and rapidly evolving. As such, they are not within the scope of this guidance.

SR 26-2, *Revised Guidance on Model Risk Management*, 17 April 2026; footnote 3 of the attached guidance. Interagency: Federal Reserve, OCC, FDIC. Supersedes SR 11-7 and SR 21-8. OCC counterpart Bulletin 2026-13.

It isn't a pass. The next sentence hands it back to your own risk management and governance practices, and the one after that says the principles here do apply to traditional models and to non-generative, non-agentic AI. The agencies have said a request for information on generative and agentic AI is coming. So whatever you build between now and then is what you'll be describing when it lands.

**We've been managing
indeterministic operators
for a very long time.**

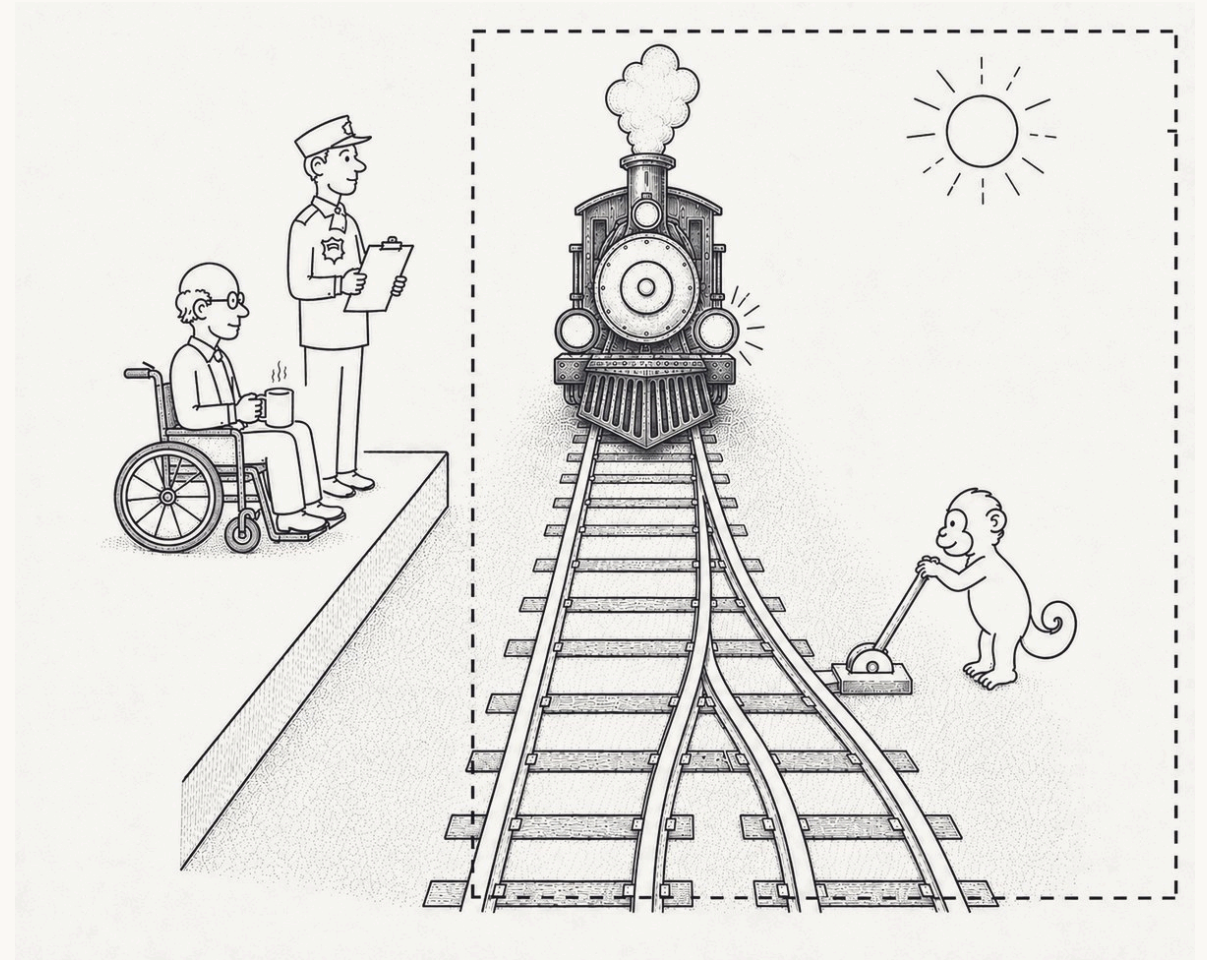
Longer than model risk management
has existed as a function.

1880S · UITENHAGE, SOUTH AFRICA

A railway hired a baboon to throw the signal levers.

James Edwin Wide lost both legs in a fall between moving railcars and came back to work as a signalman. He bought a chacma baboon at the Uitenhage market, called him Jack, and trained him to work the levers in the signal box.

A passenger looked out, saw a baboon deciding where their train went, and complained. The railway sent someone to check.

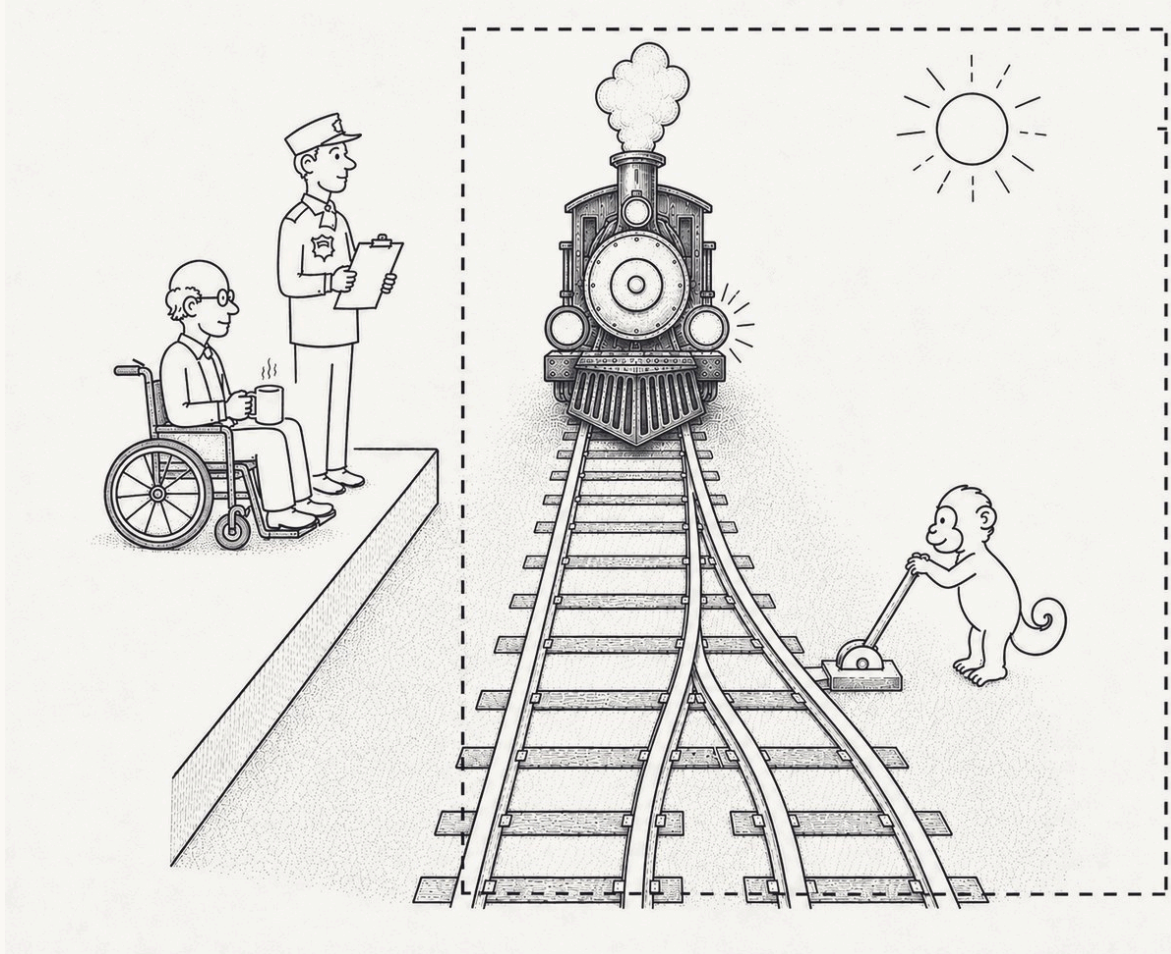


Sources: Rev. George Howe (1890) and F. W. FitzSimons, *Cape Mercury*, 29 May 1923, both cited in Nisbet, *Nature* 347:704 (1990). FitzSimons rests on an interview with Wide plus twenty-five witness statements held at the Port Elizabeth Museum. "Nine years without an error" is tradition, not record, and the inspection scene that follows is a reconstruction.

Managing baboon risk in the 1880s looks a lot like managing agent risk today.

You can't open it, and you can't ask it why in terms you
can act on. So somebody watches what it does,
decides whether they'd have done the same, and can
stop it if they wouldn't.

THE SAME THREE CHECKS, IN THE 1880S



- 1ST LINE** **Control.** Wide watches Jack work, every train, and he can stop him.
- 2ND LINE** **Validate.** Somebody who didn't train Jack forms their own view of what the levers should do, and tests him against it.
- 3RD LINE** **Audit.** Written records, which is the only reason we can argue about Jack a hundred and forty years later. Though they were collected decades after the fact, which is exactly the weakness a third line is supposed to remove.

All three checks have to run where the work runs.

There is no gate left to put them at, so they can't be meetings and they can't be documents. They have to run inside the loop, at the speed the loop runs.

FIRST LINE

GUARDRAILS

Which tools, which data, which model, and what an agent can touch. Enforced at execution rather than written in a document.

SECOND LINE

INDEPENDENT TESTS

Evaluations that form their own expectation from the spec and run on a schedule, owned by somebody who didn't build it.

THIRD LINE

THE RECORD

Every turn: what was expected, what ran, what was checked, who accepted it, and when they accepted it.

WHAT AN AGENT CAN AND CAN'T OWN

AN AGENT CAN

Do the labour of a check. Run the test, diff the output against the expectation, assemble the evidence, flag the exception. That work is mechanical, and doing it by hand is why coverage is thin everywhere.

Which is also the answer to the throughput problem. Nobody can assemble evidence at machine speed, but a person can sign what an agent assembled, and that signature is the part the loop actually needs from them.

AN AGENT CAN'T

Answer for the acceptance. That's the definition from four slides in, and it doesn't bend. Somebody who can be held accountable has to be the one who accepts.



demo-control.mp4

DEMO · FIRST LINE

Control agents with agents

Guardrails that fire at execution time rather than policy in a document. This is Wide watching every train, except it doesn't get tired.

GOVERNANCE
CONTROL
TOWER

Homepage

MRM Intake

Use Case Intake

Model Inventory

Model Graph

Findings

Audit Trail

Monitoring

Compliance

Reports

Settings

Powered by Domino

Model Inventory → auto-pricing-model-v2

```
$ adversarial-review --model "auto-pricing-model-v2" --red-team
Loading model context, code signals + governance state...
Lens 1/3 - interrogating model context & documentation...
Lens 2/3 - inspecting code & reproducibility signals...
Lens 3/3 - cross-checking governance guidelines (SR 26-2)...
> agent working... 4s
```

MODEL DESIGN, DOCUMENTATION & DATA

1 Model Materiality & Complexity Required Assess Running... Run agent

> Pull the model's risk tier, count versions/features/dependencies, and score materiality and complexity against the framework's rubric.

validation-agent · \$1 Model Materiality & Complexity RUNNING

```
$ validate --section 1 --model "auto-pricing-model-v2"
Resolving registered model + governance bundle...
Pulling model platform context (versions, deployments, evidence)...
Loading documentation, evidence answers + rubric...
> agent working... 1s
```

2 Model Documentation Quality Required Assess Not run Run agent

3 Data Quality Required Test Not run Run agent

4 Data Augmentation If applicable Test N/A

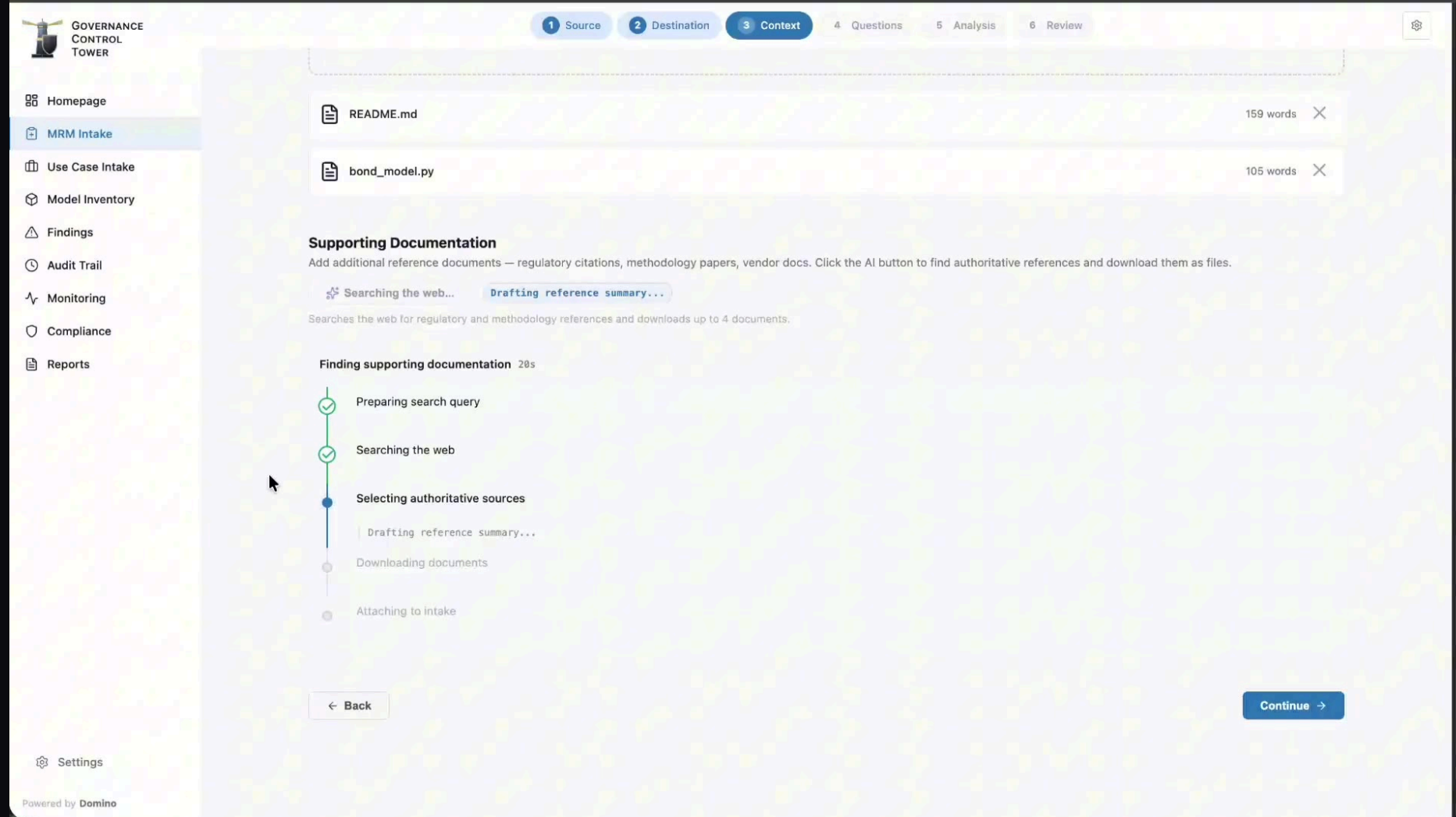
5 Model Integrity & Security Required Test Not run Run agent

6 Ethics & Legal Requirements If applicable Test Not run Run agent

DEMO · SECOND LINE

Validate agents with agents

An evaluation that forms its own expectation from the spec rather than the output, runs on a schedule, and reports to somebody who didn't build it.



DEMO · THIRD LINE

Audit agents with agents

The record as a byproduct of the loop, not a project afterwards: what was expected, what ran, what was checked, who accepted it.

Control, validate and audit the loop.



Same shape the railway used. What's different is that it runs at the speed the work runs, and it writes its own record on the way past.

BACK TO WHERE WE STARTED

When do we add a human?

Every loop whose output somebody would have to answer for. Human in the loop isn't a tier and it isn't a ceremony, it's a check sized to a specific risk. If you can't name the risk a gate closes, it isn't a control yet.

Where do we add a human?

At Check, with enough authority to Act on what they find. Let the agent do the labour of the check, and keep a named person on the acceptance, recorded before the output exists.

Questions.

Nicholas J. Goble, Ph.D. · Domino Data Lab

The Model Operator (2026) · *The Trust Functional* (2026)